

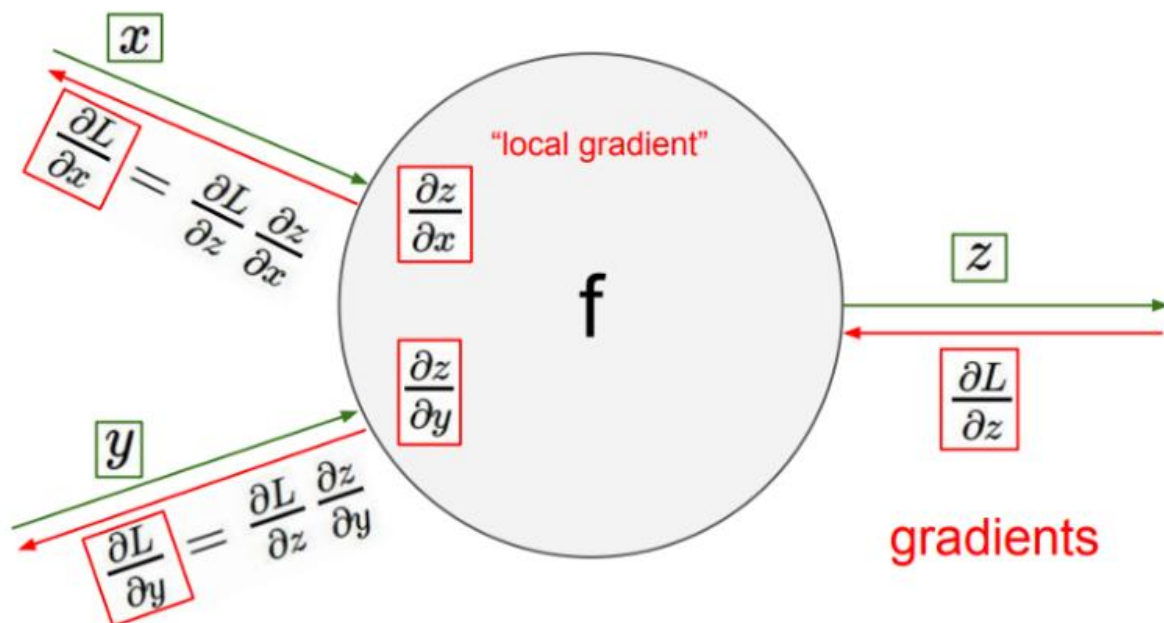
EXERCISE I

In reference to the Stochastic Gradient Descent (SDG) algorithm

I.a) Why is it useful to use continuous activation functions such as RELU as opposed to discontinuous functions (e.g. the step function)?

Discontinuous functions, like the Heaviside, are not used in CNNs for one main reason: neurons are trained by gradient descent with backpropagation, so the activation functions should be continuous and not constant in order to provide updates. In contrast, step functions have non differentiable points (discontinuity points, for example: $x=0$ in Heaviside) and the derivative is 0 elsewhere (function = constant). Therefore, using step functions as activations would not allow to update the weights through backpropagation.

I.b) Assume that the gradient value at the output of a node in a graph is $G_o=2$. The function of the node is $o=i_1^2+2i_2^2$. Compute the gradient value G_{i_1} , G_{i_2} backpropagated to the two inputs if the two inputs i_1 , i_2 have value 3 and 4, respectively.



Source: Stanford <https://www.youtube.com/watch?v=i94OvYb6noo> (also in lecture slides)

Kown variables:

$$i_1 = x = 3$$

$$i_2 = y = 4$$

$$z = i_1^2 + 2i_2^2$$

$$\frac{\partial L}{\partial z} = G_o = 2 \text{ (by text)}$$

Solution:

$$G_{i1} = \frac{\partial L}{\partial i1} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial i1} = 2 * 2i_1 = 2 * (2 * 3) = 12$$

$$G_{i2} = \frac{\partial L}{\partial i2} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial i2} = 2 * 4i_2 = 2 * (4 * 4) = 32$$

I.c) Assume that your minibatch size is 1024 samples, and training the neural network takes one week. How much time are you expecting to save if you reduce minibatch size to 256 samples and you maintain the same number of iterations on a single core cpu? What could be the undesirable effect of a smaller minibatch on the training?

Iteration: 1 batch of data processed (1024).

Epoch: full dataset processed.

- N° iterations in 1 epoch = dataset size / batch size

The size of mini-batches is essentially the frequency of updates: the smaller minibatches the more updates.

Answer1:

By keeping the same number of iterations, the training time scales linearly with the size of the minibatch, since we are reducing the total number of training epochs.

In an ideal case, without having any overhead, reducing the batch size by a factor of 4 leads a total training time of $\sim (7*24)h / 4 = 42h$, ~ 4 times lower.

Answer2, cons:

- With fewer epochs you should check the network convergence at the end of the training. A smaller batch size reduces the time for iteration, but the model is more likely to converge to a local minima.

EXERCISE II

Consider a convolutional neural network layer with the following characteristics: Consider a convolutional neural network layer with the following characteristics: 128 input channels, input_w=66, input_h=32, convolutional filters size 3x3, 128 output channels. No padding and stride 2 in the x direction, stride of 1 in the y direction.

II.a) Compute the number of MAC operations needed compute a single output channel by applying convolution on a single input channel.

$$\text{MAC} = K * K * 32 * 30 = 3 * 3 * 32 * 30 = 8'640 \text{ MACs}$$

II.b) Compute the total number of MAC operations needed to compute all the output channels in the layer. Compute the dimension of the output feature map.

$$W_{out} = \text{floor}\left(\frac{W_{in} - k + 2P}{S} + 1\right) = \text{floor}(32.5) = 32$$

$$H_{out} = \text{floor}\left(\frac{H_{in} - k + 2P}{S} + 1\right) = 30$$

$$\text{MAC} = K * K * C_{in} * C_{out} * W_{out} * H_{out} = 3 * 3 * 128 * 128 * 32 * 30 = 141'557'760 \text{ MACs}$$

$$\text{Dimension } 32 \times 30 \times 128 \times 4 = 491'520 \text{ bytes}$$

II.c) Compute the number of operations needed to perform average pooling (3x3) on all the output feature map.

Let's assume stride 1, no padding

You need K*K-1 sum +1 division operations for each pooling window

operations per window = (3*3-1) SUM + 1 DIV = 8 SUM + 1 DIV = 9

output feature map = 30x28x128

operations_per_1_output_channel = (9)*30*28=7560

operations_for_all_output_channel = (3*3)*30*28*128 = 967680

EXERCISE III

A parallel cluster of RISC-V processors is executing a convolutional layer with 8 input channels and 17 output channels. Input spatial size is 40x40. Inputs, outputs and weights are quantized to 8 bits. Convolutional filter size is 5x5, padding with zeros is used, stride=1, no bias. RELU non-linearity is applied in place on the output. After RELU, results are average-pooled (2x2 pooling window). Assume that on average a RISC-V processor can perform a MAC in 3 cycles and a RELU operation in 5 cycles and a division in 10 cycles). The three kernels (convolution, relu, average pooling) are parallelized over the output channels. No parallelization overhead

III.a) Compute the L1 minimal shared memory size that is needed to store the complete layer (input and output channels as well as weights) in L1

Let's assume avg-pooling with stride 1

Solution 1: let's assume the max pooling operation was done in place

$$W_{out} = \frac{W_{in} - k + 2P}{S} + 1 = 40$$
$$H_{out} = \frac{H_{in} - k + 2P}{S} + 1 = 40$$

Total L1 = In + out+ weights = $[40*40*8 + 40*40*17 + 5*5*8*17]$ B = 43400 B

NB: I assume no bias, otherwise Total L1 = $[40*40*8 + 40*40*17 + 5*5*8*17+17]$

Solution 2: let's assume the max pooling operation was NOT done in place

Total L1 = Convolution_memory + pooling_memory = (In_conv + out_conv+ weights_conv) + (out_pool)= 43400 + $(40-1) * (40-1) * 17$ = 69257 B

III.b) Compute the total number of cycles to process the layer on a 4-core cluster

MAC_single_output(4cores) = $K*K*C_{in} * \text{round_ceil}(C_{out}/4) = 5*5*8*5 = 1000$ MAC

MAC_all_output(4cores) = MAC_single_output(4cores) * $W_{out}*H_{out} = 1000* 40*40 = 1'600'000$ MAC

Cycles_for_MAC(4cores) = MAC_all_output(4cores) * 3 cycles = $1'600'000 * 3$ cycles = 4800000 cycles

Relu_ops (4cores) = $\text{round_ceil}(C_{out}/4)*W_{out}*H_{out} = 5*40*40 = 8'000$

Cycles_RELU(4cores) = Relu_ops * RELU_cycles = $(8'000*5) = 40'000$ cycles

Let's assume stride=1 for the pooling, no padding.

$$\text{AvgPool_ops (4cores)} = \text{round_ceil}(\text{Cout}/4) * (\text{Wout}-1) * (\text{Hout}-1) * (3\text{SUM}+1\text{DIV}) = 5 * 39 * 39 * (3\text{SUM}+1\text{DIV}) = 7605 * (3\text{SUM}+1\text{DIV})$$

$$\text{Cycles_AvgPool(4cores)} = \text{AvgPool_ops} * \text{AvgPool_cycles} = 7605 * (3\text{SUM}+1\text{DIV}) = 7605 * (3 * 3\text{cycles} + 10\text{cycles}) = 144'495 \text{ cycles}$$

$$\text{Cycles_Tot(4cores)} = \text{Cycles_for_MAC(4cores)} + \text{Cycles_RELU(4cores)} + \text{Cycle_AvgPool(4cores)} = 4'800'000 + 40'000 + 144'495 = 4984495 \text{ cycles}$$

III.c) Compute the speedup obtained when processing the layer with 4 cores versus only 1 core (sequential).

1) Let's first calculate the sequential processing (1 core)

$$\text{MAC_single_output(1core)} = K * K * \text{Cin} = 5 * 5 * 8 = 200 \text{ MAC}$$

$$\text{MAC_all_output(1core)} = \text{MAC_single_output(1core)} * \text{Cout} * \text{Wout} * \text{Hout} = 200 * 17 * 40 * 40 = 5440000 \text{ MAC}$$

$$\text{MAC_Cycles(1core)} = \text{MAC_all_output(1core)} * 3 = 16320000$$

$$\text{Relu_ops} = \text{Cout} * \text{Wout} * \text{Hout} = 17 * 40 * 40 = 27200$$

$$\text{RELU_Cycles(1core)} = \text{Relu_ops} * 5 = 136000$$

$$\text{AvgPool_ops (1cores)} = \text{Cout} * (\text{Wout}-1) * (\text{Hout}-1) * (3\text{SUM}+1\text{DIV}) = 17 * 39 * 39 * (3\text{SUM}+1\text{DIV}) = 25857 * (3\text{SUM}+1\text{DIV})$$

$$\text{Cycles_AvgPool(1core)} = \text{AvgPool_ops} * \text{AvgPool_cycles} = 25857 * (3\text{SUM}+1\text{DIV}) = 25857 * (3 * 3\text{cycles} + 10\text{cycles}) = 491283 \text{ cycles}$$

$$\text{Cycles_Tot(1core)} = \text{MAC_Cycles(1core)} + \text{RELU_Cycles(1core)} + \text{AvgPool_ops (1core)} = 16320000 + 136000 + 491283 = 16947283$$

$$\text{Speedup: Cycles_Tot(1core) / Cycles_Tot(4cores)} = 16947283 / 4984495 = 3.4x$$